AI Evaluation

Xây hệ thống evaluation cho sản phẩm AI: dataset, trace analysis, code evals, LLM judge, human review

AI IN ACTION — DAY 20–21 — TRACK 1

Instructor: Mai Anh Nguyen (Blue)

Agenda

01	**Đọc trace**	Transcript là thứ user thấy — trace là thứ PM phải đọc, từng bước agent thực thi.
02	**Thiết kế dataset có coverage**	User Input Grid: LLM giúp sinh input, con người kiểm soát coverage và review lại.
03	**Cảm tính → tiêu chí đo được**	Thu hẹp scope, định nghĩa "đủ tốt", formalize thành rubric cả team chấm giống nhau.
04	**Chọn evaluator cho từng tiêu chí**	Cái gì kiểm bằng code, cái gì cần LLM judge, cái gì phải đến tay expert.
05	**Đưa expert vào loop**	Expert định nghĩa chuẩn vàng — dùng để calibrate judge và bắt failure mode mới.



INSTRUCTOR

Mai Anh Nguyen (Blue)

Generalist Product Builder

2026	FPT Long Châu · PM, Healthcare Product
2025	Thongtincuuho.org · Co-founder
2025	FPT Software AI Center · PM, AI Agent
2021–2025	Xantus · PM, On-chain Analytics & AI Agent
2016–2021	DYNO, Kalapa · PM, OCR & eKYC & Credit Scoring

LinkedIn | Facebook

BỐI CẢNH

Câu chuyện quen thuộc của mọi team làm AI

NGÀY DEMO

Demo chạy tốt

Khách hàng sớm khen, stakeholder đồng thuận. Bạn quyết định mở rộng.

→

VÀI TUẦN SAU

Bức tranh mờ dần

User đông hơn: người rất thích, người phàn nàn, người không quay lại. Đổi model — không biết còn ổn không.

→

HÔM NAY

Product Review ngày mai

Leadership hỏi bạn track chất lượng bằng cách nào. Usage metrics không trả lời được.

"AI của mình có đang làm việc tốt một cách ổn định không?"

— bạn không có câu trả lời. Đó là evaluation gap.

Vai trò của PM đã mở rộng thêm phân bố chất lượng

TRADITIONAL PRODUCT

Deterministic — đúng hoặc sai

Input A	→	Output B	✓ Pass
Input A	→	Output B	✓ Pass
Input A	→	Output B	✓ Pass

Chạy bao nhiêu lần cũng y hệt — kiểm thử chỉ có hai kết quả: 1 hoặc 0.

METRIC **Test Pass / Fail — 1 hoặc 0**

Thế giới deterministic — cùng input, cùng output; test pass hay fail.

AI PRODUCT

Wide outcome distributions



METRIC **Agent Success Rate**

Thế giới probabilistic — cùng input, output vẫn khác nhau.

AI Evals có hai lớp — product team làm lớp trên

PRODUCT TEAM · LÀM Ở ĐÂY

Application Evals

Chất lượng sản phẩm trong ngữ cảnh user thật

- Accuracy
- Helpfulness
- Tone / Style
- Safety
- Latency
- Format compliance
- User intent match
- Cost efficiency

MODEL PROVIDER

Model Evals

Benchmark chuẩn hoá cho năng lực nền

- Reasoning
- Coding
- Math
- Instruction following
- Safety

Model giỏi benchmark không đảm bảo sản phẩm đạt chất lượng — hai lớp đo hai thứ khác nhau.

CASE STUDY MỞ MÀN

Notion AI

Một team coi evals là IP của mình.

Notion AI dành 90% thời gian cho evals

Chỉ 10% là viết prompts.

100M+

users toàn cầu

Scale khổng lồ — không thể test bằng tay hay intuition.

60%

users không nói tiếng Anh

100% engineer nói tiếng Anh — buộc phải dựa vào evals.

< 1 ngày

để ship model mới

Evals tự động cho biết ngay prompt nào regression khi model mới ra.

"Evals is our IP — everything we build, how we decide if those work well, lives inside Braintrust."

Sarah Sachs — AI lead, Notion

Nguồn: How to build world-class AI products — Sarah Sachs (AI lead @ Notion) & Carlos Esteban (Braintrust)

Evals là user research mới của PM

"You can think of this as your version of User Research." Sarah Sachs — AI lead, Notion

USER RESEARCH TRUYỀN THỐNG

User interviews
Hỏi user nghĩ gì — không phải user thật sự làm gì.

Surveys
Số lượng lớn nhưng context nghèo — khó biết tại sao user không hài lòng.

Chậm và tốn kém
Vài tuần để có insight — không kịp với tốc độ release AI.

EVALS AS USER RESEARCH — CÁCH NOTION LÀM

Thumbs-down data
Natural-language request thật từ user — không phải câu hỏi PM đặt ra trước.

Behavior thật, không phải claimed behavior
User muốn draft report, không chỉ research — tìm ra từ data, không từ interview.

Real-time, liên tục
PM ngồi trong eval tool mỗi ngày — insight cập nhật theo từng release.

PM không cần viết scoring function — nhưng phải là người đọc data và escalate findings. Việc này không ai làm thay được.

Notion quản lý evals như thế nào — ai làm gì

BƯỚC	PRODUCT MANAGER	ENGINEER	DATA SPECIALIST
1 · Decide Xác định vấn đề cụ thể	Đọc thumbs-down data và usage log để tìm điểm cần cải thiện.	Xác định cải tiến cần làm — ví dụ thêm một connector mới.	—
2 · Curate Tạo targeted dataset	Kiêm luôn vai data specialist ở team nhỏ.	—	Tạo dataset từ log thật. Bắt đầu với 10 samples, format đúng trước.
3 · Score Gắn scoring functions	—	Viết scoring function sau khi đã nhìn data lâu. Mix LLM-as-judge và heuristic.	Viết judge prompt riêng cho từng sample — tốn công nhưng sâu hơn nhiều.
4 · Inspect Review failures kỹ	Phát hiện pattern thật, rồi escalate findings lên roadmap.	Đưa failures vào một LLM để tóm tắt theme, ra báo cáo nhanh.	Human label failures. Quality quan trọng hơn quantity.

↺ Loop trở lại bước 1: chỉnh prompt theo failure theme, thêm rows dataset từ log mới.

Data lấy từ đâu, và dùng theo nguyên tắc nào

LẤY DATA TỪ ĐÂU

1 Dog-fooding — data tự nhiên

Team dùng sản phẩm của mình hàng ngày, nên mọi interaction đều là data tiềm năng. Không cần giả lập hay synthetic data từ đầu.

2 Thumbs-down — signal từ user

Chỉ lấy natural-language request, không lấy output cũ vì output lỗi thời rất nhanh. Request thật của user là tài sản dài hạn, dùng lại được mãi để eval model mới.

BA NGUYÊN TẮC KHI DÙNG DATA

Bắt đầu nhỏ: 10 samples, không phải 1000
Format đúng trước — dummy data sai cấu trúc là sai từ gốc.

Quality quan trọng hơn quantity
Đặc biệt khi fine-tuning — ít sample tốt hơn nhiều sample kém.

Data flywheel tích lũy theo thời gian
Mỗi release thêm log mới, dataset lớn dần, evals ngày càng chính xác hơn.

"The people closest to the data should be the ones creating the dataset."

Sarah Sachs — AI lead, Notion

Có hệ thống evals thì thay đổi những gì

× Trước

Dùng spreadsheet để label data
Human labeler quá tải, phải parse prompt bằng công thức bảng tính.

Model mới cần vài tuần để validate
Không biết prompt nào regression — phải test thủ công từng cái.

Ship dựa trên gut feeling
Chạy tốt trong demo — không biết có nhất quán với user thật không.

✓ Sau — với hệ thống evals

Model mới ship trong một ngày
Chạy toàn bộ evals, biết ngay regression, đổi model tự tin.

Tự tin build cho user đa ngôn ngữ
Multilingual evals đảm bảo chất lượng cho ngôn ngữ engineer không nói được.

Insight sản phẩm đến từ data
User muốn draft report — phát hiện từ thumbs-down, không từ interview.

"I don't think that we could exist without a tool like this today — it is critical to our iteration flow, and it actually is our IP."

Sarah Sachs — AI lead, Notion

PART 01

AI Evals Lifecycle

Từ quan sát thủ công đến kiểm soát chất lượng liên tục.

Evals là một vòng lặp, không phải một lần kiểm thử

0
Chuẩn bị
PHA PROTOTYPE

Product promise
Unit of evaluation
Instrument agent
Scenarios ban đầu

1
Vibe Check
PHA PROTOTYPE

Chạy case nhỏ
Đọc output và trace tay
Tìm failure modes
Định nghĩa "đủ tốt"

2
Offline Evals
PHA BUILD

Formalize rubric
Xây reference dataset
So sánh candidates
Ship / Limited / Hold

3
Production Monitoring
PHA PRODUCTION

Theo dõi quality và cost
Sample traces để review
Phát hiện drift
Điều tra incidents

↺ **Continuous Improvement** failure thật thành dataset · cập nhật rubric · regression test · lặp lại lifecycle

"Evals không phải một lần kiểm thử trước release — đó là vòng lặp biến quan sát thực tế thành quyết định sản phẩm có thể kiểm chứng."

Tham chiếu: Anthropic — demystifying evals for AI agents

Bước 1 — Vibe Check: chạy trước khi viết PRD

Mục tiêu là khám phá behavior, chưa phải chấm điểm chính thức.

1

Generate inputs

10–30 đầu vào bao phủ nhiều persona và use case.

happy path | edge case

2

Run prototype

Chạy từng đầu vào qua prototype, quan sát model vận hành thật.

input → agent → output

3

Label outputs

Gắn nhãn từng đầu ra, ghi lý do pass hoặc fail.

Ship | Chỉnh | Fail

4

Note patterns

Model làm được gì, không làm được gì, hành vi nào bất ngờ?

what works | surprises

FAIL LẶP LẠI → SEEDS YOUR PRD

Những gì model không làm được định hình chính xác những gì PRD cần đặc tả.

PASS TỐT NHẤT → SEEDS GOLDEN OUTPUTS

Output tốt nhất từ vibe check trở thành reference dataset cho few-shot và evals.

Demo before memo — quan sát model trước, viết yêu cầu sau.

Output bước 1 — Golden Outputs

Reference dataset là câu thật cộng nhãn đúng kỳ vọng — dùng cho few-shot và cho evals.

#	CÂU NGƯỜI DÙNG	Ý ĐỊNH	CẢM XÚC	ƯU TIÊN
1	Mình nhận áo rồi mà chật quá, đổi size sao vậy?	Đổi hàng	Khó chịu	Trung bình
2	Shop giao nhầm màu cho mình rồi	Đổi hàng	Bực bội	Cao
3	App báo giao rồi mà mình chưa thấy đơn đâu	Tra cứu đơn	Tức giận	Cao
4	Mình muốn trả đôi giày hôm trước mà không còn mã đơn nữa	Trả hàng	Lo lắng	Trung bình
5	Cái đơn hôm bữa ấy, thôi giờ mình không lấy nữa được không	Cần làm rõ	Trung tính	Trung bình

PHẢI CÓ EDGE CASE

Câu mơ hồ · Thiếu thông tin · Nhiều ý trong một câu · Không nhớ mã đơn

"Áo không giống hình lắm, giờ đổi hay trả được ta"

"Mình đặt 2 đơn, 1 đơn muốn trả, 1 đơn muốn đổi size"

"Cái đơn hôm trước đó, check giúp mình với"

- ✓ Case mơ hồ giúp test khả năng hỏi lại
- ✓ Không ép model đoán khi chưa rõ
- ✓ Dataset tiếp tục lớn dần sau production

Bước 2 — Offline Evals: chấm dataset trước release

01 **Trigger thay đổi**
Prompt mới · model upgrade · tool integration

↓

02 **Chạy trên reference dataset**
Tự động, không cần manual review

↓

03 **So với baseline**
Version hiện tại là chuẩn so sánh

Deploy
Quality ok, approve

Fix & iterate
Regression — điều tra rồi chạy lại

Sau khi fix, quay lại bước 01 và chạy lại toàn bộ eval.

BENCHMARK CỐ ĐỊNH — MỌI RELEASE DÙNG CHUNG

Reference dataset

50 historical tickets, labels đã verify

Categorization accuracy eval

Sentiment precision eval

Escalation recall eval

Critical — không được phép regression

CẬP NHẬT DATASET KHI

Phát hiện failure mode mới từ production

Scope sản phẩm thay đổi

Evals quá dễ pass

"Nếu không thể ship model mới ngay trong ngày ra mắt — bottleneck là evaluation, không phải model."

Một kết quả eval nhìn như thế nào

DIMENSION	BASELINE	CANDIDATE	DELTA
Accuracy % output đúng rubric	71%	**78%**	+7%
Latency P50 (median) response time	1.2s	**1.8s**	+0.6s
Edge case pass rate % edge case xử lý đúng	44%	**61%**	+17%
Regression count Case từng pass, nay fail	—	**3**	Cần review
Quality gate Pass threshold ≥ 75%	71% — fail	78% — pass	Gate cleared

VERDICT

Ship

Accuracy +7%, edge case +17%, gate đã qua.

CẦN ĐỌC THÊM

3 regression traces — hiểu tại sao fail trước khi ship.

Số liệu cho phép ship. Nó không thay việc đọc trace.

Đừng chỉ nhìn con số tổng — **trung bình che khuyết điểm**: tổng score tăng 82%→85% vẫn có thể giấu một phân khúc tụt 70%→55%. Luôn đọc theo slice và mở regression traces trước khi ship. Và trên dataset nhỏ: **50 dòng → 1 trace lật = 2 điểm %** — chênh lệch ≤2% là nhiễu, không phải cải thiện.
Cần bao nhiêu case để tin một khoảng chênh (95%)? **Chênh 10% → ~100 · 3% → ~1.000 · 1% → ~10.000** (OpenAI) — chênh nhỏ đi 3×, số mẫu cần tăng 10×.

Bước 3 — Sau launch, user tạo ra case bạn chưa nghĩ tới

SIGNAL MỚI TỪ PRODUCTION

Intent mới
User dùng AI theo cách team chưa nghĩ tới khi thiết kế

Ngôn ngữ và data format lạ
Tiếng địa phương, ký tự đặc biệt, input không chuẩn

Behavior ngoài use case ban đầu
User kéo AI về tình huống không nằm trong PRD

Tool và data source thay đổi
API third-party update, schema đổi, pipeline drift

User expectation thay đổi
Chuẩn "tốt" của user dịch chuyển theo thời gian dùng

Q1

Product còn đạt quality bar ngay lúc này không?

Agent Success Rate trong production có dưới ngưỡng?

REAL-TIME CHECK

Q2

Offline và online score có đang diverge?

Offline tốt mà online xấu — dataset chưa phản ánh thực tế.

DISTRIBUTION GAP

Q3

Failure mode nào mới xuất hiện?

Unknown unknowns — offline eval không thấy được cho tới khi có user thật.

NEW BLIND SPOTS

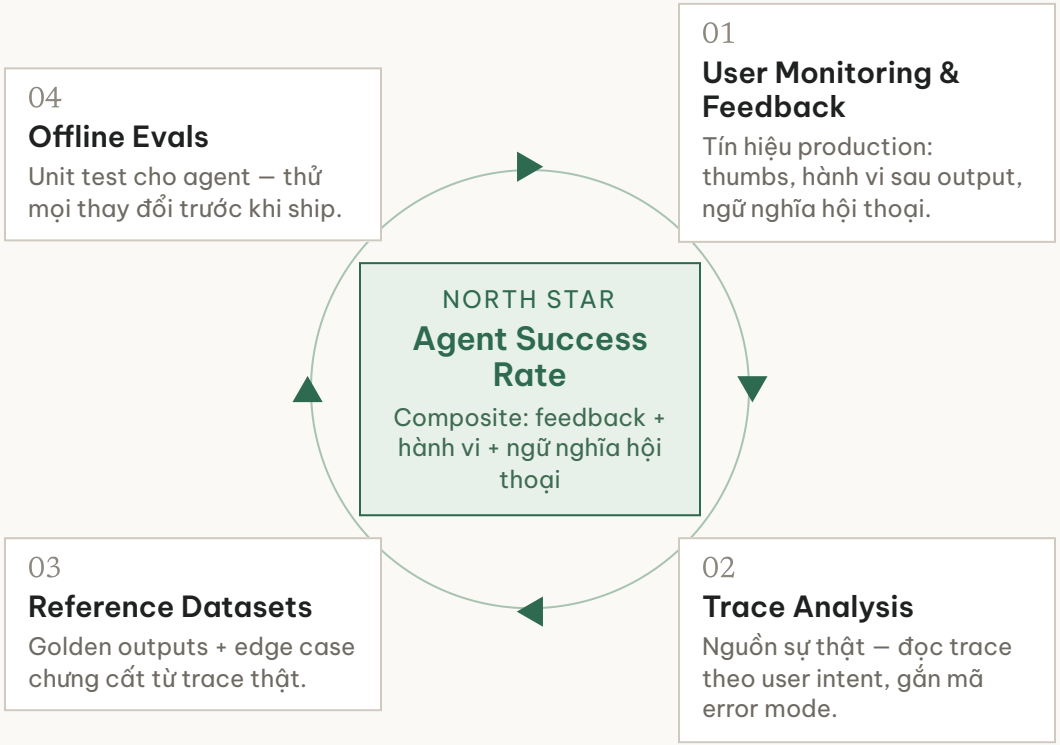
Q4

User feedback có mâu thuẫn với eval?

Thumbs-down tăng mà eval score ổn — rubric đang đo sai thứ.

SIGNAL CONFLICT

AI Flywheel — bánh đà cải thiện liên tục sau launch



"Những sản phẩm AI tốt nhất không tự nhiên đã tuyệt vời ngay từ đầu — họ xây một flywheel để liên tục cải thiện theo thời gian."

AI Flywheel Framework · Calibre Labs

PRD cho sản phẩm AI cần thêm bốn mục — và một mục mới hẳn

TRADITIONAL PRD		AI-NATIVE PRD BỔ SUNG	TẠI SAO CẦN THÊM
Goals & success metrics DAU · Conversion · Retention	+	**Evaluation Rubric** Accuracy %, latency, pass/fail rubric	AI không có conversion rate — cần rubric riêng để đo chất lượng output.
User stories / use cases Luồng người dùng · Acceptance criteria	+	**Golden Outputs** Input → expected output cụ thể, test được ngay	"As a user, I want" không test được — cần output mẫu thật để eval.
Functional requirements Feature list · Business logic	+	**Prompt Logic & Tools** System instructions, API tools, failure rules	AI cần biết cách xử lý edge case — không chỉ "tính năng là gì".
Open questions Câu hỏi chưa rõ	+	**Dataset Strategy** Unknowns → dataset cần thu thập để eval	"Chưa rõ" phải thành "cần thu thập dataset này".
(không có tương đương)	MỚI	**Edge Case Handling** Khi model fail — thất bại thế nào là chấp nhận được?	AI fail theo nhiều cách — PRD phải định nghĩa "fail gracefully" nghĩa là gì.

"PRD định nghĩa quality bar – không chỉ danh sách tính năng."

Một AI PRD hoàn chỉnh — Support Ticket Triage v1.0

Sample AI PRD — v1.0 Status: Prototyping

1. **Problem & Business Value** — 4 giờ/ngày tag tay → agent phân loại real time
2. **Prompt Logic & Dataset** — system instruction + 50 golden tickets đính kèm
3. **Tool Specification** — 4 API agent được phép gọi, và để làm gì
4. **Evaluation Criteria** — target đo được cho từng metric
5. **Edge Cases Handling** — fallback khi model không chắc chắn
6. **Prototype & Early Findings** — những gì chạy thử dạy cho spec
7. **Technical Constraints** — PII scrubbing, cost < $0.01/ticket

▪ Mục 4–6: phần không tồn tại trong PRD truyền thống

Quality bar viết thành số, không phải tính từ

Accuracy > 92% | Sentiment > 85% | Latency < 2s | Hallucination = 0%

Fail gracefully được thiết kế trước

Confidence < 0.7 → không auto-tag, chuyển Needs Human Review · audit tay 5% ticket mỗi tuần · nút "Tag này đúng không?" cho support.

PRD viết SAU khi prototype

Findings thật quay lại sửa spec: input quá ngắn gây hallucinate → thêm chế độ hỏi lại · 20% ticket có 2 issue → cần multi-label · sarcasm → thêm 5 ví dụ vào golden dataset.

Định nghĩa tiêu chí chất lượng

Thu hẹp cho đến khi viết được rubric Pass/Fail mà hai người chấm độc lập cho cùng kết quả.

QUÁ RỘNG — KHÔNG ĐO ĐƯỢC

“Cải thiện trải nghiệm hỗ trợ khách hàng bằng AI”

↓ **Trục 1 · Tách theo task**

Phân loại ticket · soạn trả lời · quyết định escalate · cập nhật CRM

↓ **Trục 2 · Tách theo điều kiện input**

Ticket rõ ràng · ticket mơ hồ — kỳ vọng khác: hỏi lại thay vì đoán

↓ **Trục 3 · Tách theo mức tự trị**

UNIT OF AI WORK

Phân loại 1 ticket → đúng 1 trong 3 nhóm, kèm sentiment, dưới 2 giây

✓ Viết được rubric Pass/Fail — sẵn sàng đo

PART 02

Thiết kế coverage & candidate scenarios

Chọn biến làm agent phải hành xử khác đi, trước khi viết test prompt.

Vấn đề của việc “hãy tạo 50 test prompts”

Dataset nhiều rows chưa chắc có coverage tốt.

× LLM-generated dataset

Trông nhiều — thực ra rất đồng nhất.

happy path A	variant 1	near-dup
happy path B	variant 2	duplicate
happy path C	duplicate	near-dup

50 rows ≈ 3 real cases

vs

✓ Real production dataset

Ít rows hơn — coverage thật sự.

edge case	persona B	happy path
ambiguous	biz rule	multi-intent
real trace	zero-hit	stale source

20 rows = 20 real cases

Đo được: xin LLM 4.000 ý tưởng về một chủ đề → chỉ còn **~200 ý không trùng lặp** — khả năng tự sinh đa dạng bão hoà rất nhanh (Si & Yang 2024).

Thuộc tính nào là dimension? Nhìn vào cột hành vi đúng

Cùng một phép thử cho hai thuộc tính của input: đổi giá trị → hành vi đúng có đổi không?

DIMENSION? Cách diễn đạt — ✕ Không – bỏ

INPUT – 3 GIÁ TRỊ CỦA THUỘC TÍNH	HÀNH VI ĐÚNG
GIÁ TRỊ: LỊCH SỰ "Tôi muốn hoàn tiền đơn #4521."	→
GIÁ TRỊ: CỘC LỐC "Refund #4521!"	→ Vẫn chỉ MỘT hành vi: kiểm tra chính sách → tạo yêu cầu hoàn
GIÁ TRỊ: CÂU HỎI "Đơn #4521 hoàn tiền được không?"	→

3 giá trị → 1 hành vi. Chỉ là paraphrase – để LLM tự sinh, không tốn một trục grid.

DIMENSION? Độ đủ thông tin — ✓ Là dimension

INPUT – 3 GIÁ TRỊ CỦA THUỘC TÍNH	HÀNH VI ĐÚNG
GIÁ TRỊ: ĐỦ THÔNG TIN "Hoàn tiền đơn #4521, giao trễ 5 ngày."	→ Xử lý luôn
GIÁ TRỊ: THIẾU MÃ ĐƠN "Hoàn tiền đơn của tôi."	→ Hỏi đúng mã đơn
GIÁ TRỊ: MƠ HỒ "Đơn vừa rồi tệ quá!"	→ Làm rõ ý định trước

3 giá trị → 3 hành vi khác nhau. Thành một trục của grid – mỗi giá trị phải có mặt trong dataset.

Đọc từ trên xuống: **Thuộc tính** (nhãn Dimension?) → **3 giá trị** (tag trên mỗi input) → **hành vi đúng**. Hành vi đổi theo giá trị → thuộc tính đó là dimension.

User Input Grid — những biến làm agent phải hành xử khác đi

Hai chiều ví dụ: Persona × User intent. Mỗi ô là một tình huống khác nhau — con người quyết ô nào đáng test.

	Hỏi khái niệm	Tra cứu bài giảng	Xin lộ trình	Khiếu nại / hoàn phí
Học viên mới	test ✓		test ✓	phi lý
Học viên giữa khoá		test ✓		
Người ôn thi	test ✓			phi lý
Manager duyệt khoá	phi lý		phi lý	test ✓

▪ Chọn test — có ý nghĩa sản phẩm ▨ Loại — tổ hợp phi lý ▫ Cân nhắc sau

Thêm chiều thứ 3, thứ 4…

Context richness · Ambiguity · Failure cost · Ngôn ngữ — mỗi chiều nhân thêm biến thể.

Ô ≠ dòng dataset

Mỗi ô chọn sẽ sinh 2–3 cách diễn đạt tự nhiên — LLM chỉ giúp ở bước đó.

Coverage = chọn ô có chủ đích

Không phải test hết 16 ô — mà biết rõ vì sao chọn ô này, bỏ ô kia.

Chọn dimension — biến nào tạo ra sự đa dạng có ý nghĩa?

Kiểm nhanh: đổi giá trị của biến → hành vi đúng của agent phải đổi theo. Không đổi → chỉ là paraphrase.

5 DIMENSION PHỔ BIẾN

ICP	Chân dung khách hàng mục tiêu: Enterprise · Mid-market · SMB
Persona	Vai trò người dùng cuối: Engineer · Manager · Executive
User Intent	Task user đang cố hoàn thành
Context Richness	Thông tin đầy đủ vs thiếu chi tiết
Ambiguity Level	Chỉ dẫn rõ ràng vs yêu cầu mơ hồ

Danh sách là điểm xuất phát — bộ dimension thật mọc từ use case của bạn.

VÍ DỤ · AI TUTOR HỎI ĐÁP TÀI LIỆU KHOÁ HỌC, CÓ TRÍCH NGUỒN

DIMENSION	OPTION 1	OPTION 2	OPTION 3
Đáp án trong tài liệu?	Phải tổng hợp nhiều bài	Có trực tiếp	Ngoài phạm vi
Người học	Học viên mới	Đã học qua	Người ôn thi
Intent	Hỏi khái niệm	Tra cứu bài giảng	Xin lộ trình ôn
Độ rõ câu hỏi	Thiếu ngữ cảnh	Rõ, đủ ngữ cảnh	Nhiều ý một câu

▪ Một tổ hợp = Phải tổng hợp + Học viên mới + Hỏi khái niệm + Thiếu ngữ cảnh

→ "Eval với test khác nhau chỗ nào nhỉ? Bài nào cũng nhắc mà em chưa hiểu" — hành vi đúng: tổng hợp từ nhiều bài, cite từng nguồn, giải thích từ nền.

Quy trình tạo User Input Grid

Thiết kế coverage trước khi viết test prompt.

BƯỚC 1

Chọn dimensions

Yếu tố nào làm expected behavior thay đổi?

- Loại câu hỏi của user
- Độ phủ tài liệu khóa học
- Có cần research ngoài không
- Ngôn ngữ và expertise

BƯỚC 2

Định nghĩa values

Đổi value thì câu trả lời đúng cũng phải đổi theo.

- Có slide trả lời trực tiếp
- Cần tổng hợp nhiều module
- Course chỉ cover một phần
- Course chưa đề cập

BƯỚC 3

Tạo combinations

Giữ lại combination khi:

- Có khả năng xuất hiện thật
- Làm agent phải đổi chiến lược
- Failure cost cao
- Team chưa chắc quality boundary

BƯỚC 4

Thêm constraint đời thật

Làm scenario bớt "sạch", giống user thật.

- Hỏi mà không rõ bài nào
- Slide và transcript dùng thuật ngữ khác
- Source cũ vẫn đứng đầu retrieval
- Answer đúng nhưng follow-up lặp lại

BƯỚC 5

Viết thành user inputs

LLM chỉ paraphrase — cùng scenario, nhiều cách diễn đạt.

- ngắn / dài
- formal / informal
- tiếng Việt / tiếng Anh
- beginner / advanced

× Không giao cho LLM tự chọn test coverage — human kiểm soát coverage, LLM chỉ diễn đạt lại.

PHỄU TỔ HỢP $4 \times 4 \times 4 = 64$ tổ hợp → loại phi lý → 15–20 scenario → ×2–3 diễn đạt → 40–50 test inputs

Từ grid tới Candidate Scenario Bank

CẤU TRÚC MỘT SCENARIO

SC-042 **Challenge**

NATURAL-LANGUAGE INPUT

“Trong bài có nói sâu về retrieval không, hay chỉ là giới thiệu?”

GRID COMBINATION

intent: clarify · coverage: partial · external-research: no · stage: mid-course

EXPECTED BEHAVIOR

Hỏi lại để xác định “bài nào”, rồi trả lời trong phạm vi tài liệu — không tự suy thêm.

WHY INCLUDED

Ambiguous reference cộng partial coverage — failure cost cao nếu agent tự đoán.

BA LOẠI SCENARIO

Representative scenarios

Gần với cách user thường hỏi — về sau phải phản ánh production distribution.

Challenge scenarios — cố ý over-sample

- Ambiguous inputs
- Source conflicts
- Retrieval zero-hit
- Multi-intent requests
- Stale external sources
- Follow-up lặp hoặc sai scope

Pass rate trên challenge set không phải production success rate.

Critical regression candidates

- Claim ngoài bị nói thành nội dung đã dạy
- Citation không support claim
- Agent reasoning từ false premise
- View source dẫn tới source không liên quan

Coverage đến từ cách chọn dimension và combination — không đến từ số lượng rows. **“If you care about something, put a test set on it.”** — Chip Huyen

PART 03

Chạy agent, đọc trace, chuẩn hoá chất lượng

Đừng bắt đầu bằng automation — đọc trace trước.

Trace là gì, và vì sao PM phải đọc trace

TRÊN MẶT NƯỚC — THỨ USER THẤY

“Your flight has been booked.”

Transcript — lời agent nói

DƯỚI MẶT NƯỚC — THỨ PM PHẢI ĐỌC

- System prompt & ngữ cảnh được nạp
- Tool calls + tham số — booking API có được gọi không?
- Retrieved documents — lấy đúng tài liệu chưa?
- Reasoning từng bước giữa các lần gọi
- State cuối — reservation có thật trong database không?

Transcript là lời kể

Agent nói “đã đặt vé xong” — nghe rất tự tin.

Trace là bằng chứng

Toàn bộ chuỗi bước thực thi — nơi duy nhất cho biết reservation có tồn tại thật.

Lỗi chìm dưới mặt nước

Retrieval sai, tool gọi thiếu tham số — không bao giờ lộ trên transcript.

Một trace đọc ra sao

trace SC-042 · 4.2s · 3 tool calls

1	USER INPUT	“Trong bài có nói sâu về retrieval không?”
2	INTENT	classified: course_content_lookup
3	TOOL: RETRIEVAL	top hit: blog post ngoài khoá học (2023) slide của bài không nằm trong 5 kết quả đầu
4	SYNTHESIZE	Tổng hợp từ nguồn ngoài, không kiểm tra lại nguồn nội bộ
5	FINAL RESPONSE	“Bài có nói khá sâu về retrieval, gồm ba chiến lược…”
6	FEEDBACK	thumbs-down: “bài không có phần này”

NẾU CHỈ ĐỌC TRANSCRIPT

Câu trả lời trôi chảy, đúng ngữ pháp, nghe có thẩm quyền — không thấy vấn đề.

ĐỌC TRACE MỚI THẤY

Lỗi bắt đầu ở bước 3: retrieval lấy nguồn ngoài. Bước 4 và 5 chỉ là hệ quả.

NÊN SỬA Ở ĐÂU

Sửa retrieval và thêm bước xác nhận nguồn — không sửa câu chữ của response.

Từ chạy thử agent đến rubric đánh giá

VÒNG 1 — KHÁM PHÁ BEHAVIOR

VÒNG 2 — CHUẨN HOÁ

STEP 1

Chạy bộ tình huống test

Tình huống phổ biến, khó, rủi ro cao. Cố định prompt, model, tool, version dataset.

Output: traces đã log

STEP 2

Đọc trace, ghi nhận tự do

Chưa chấm điểm. Ghi điều bất ngờ, lỗi rõ ràng, behavior tốt, kèm evidence cụ thể.

Output: notes thô có evidence

STEP 3

Gom nhóm pattern lặp lại

Lỗi hay mắc, điểm chất lượng yếu, behavior tốt cần giữ, nhóm scenario bị ảnh hưởng.

Output: failure / success patterns

STEP 4

Xác định lỗi bắt đầu từ đâu

Bước cuối còn đúng là gì? Bước đầu tiên sai là gì? Đâu là nguyên nhân, đâu là hệ quả?

Output: root-cause map

STEP 5

Biến pattern thành tiêu chí

Trace codes, định nghĩa pass/fail, ví dụ đúng và sai, rubric, human label có cấu trúc.

Output: trace codes + rubric v1

× Đừng chạy vài prompt ngẫu nhiên rồi gọi đó là eval — phải biết mình đang test loại tình huống nào.

Chuẩn hoá notes thành trace codes

Notes tự do sau vòng discovery

Mỗi người nói một kiểu.

“con bot này xử lý sai ý khách”

“nó hơi bị lạc hướng”

“nó không hiểu khách muốn đổi hàng”

“agent trả lời mà không tra cứu đơn”

“agent hứa hoàn tiền quá sớm”

“agent không chuyển human khi phức tạp”

Khó đo Khó so sánh Khó sửa

formalize →

Trace codes — ngôn ngữ chung của team

Cùng một vấn đề, cùng một tên.

wrong_intent
Agent hiểu sai nhu cầu chính của user

missing_lookup
Không tra cứu dữ liệu trước khi trả lời

premature_commit
Hứa kết quả khi chưa đủ điều kiện

missing_escalation
Không chuyển human khi vượt thẩm quyền

no_next_step
Đúng policy nhưng thiếu hành động tiếp theo

Đo được So sánh được Sửa được

Taxonomy phải tuỳ theo use case

Không có một taxonomy dùng chung — xây từ trace analysis của chính mình.

AI Support đơn hàng

Xoay quanh: intent · order · policy · authority

- wrong_intent
- missing_order_lookup
- policy_misapplied
- premature_refund_commit
- missing_escalation
- unclear_next_step

AI Learning Assistant

Xoay quanh: retrieval · grounding · boundary

- wrong_question_intent
- poor_retrieval
- unsupported_claim
- external_boundary_violation
- poor_followup_quality

AI Travel Planner

Xoay quanh: feasibility · data · authority

- missing_constraint
- stale_live_data
- infeasible_schedule
- budget_violation
- booking_overreach
- missing_recovery

Đừng copy taxonomy từ bài khác — dùng trace analysis để xem use case của mình thật sự fail ở tầng nào.

PART 04

Automated Evaluators

Khi nào nên automate · Code-based vs LLM-judge · Common mistakes

VẤN ĐỀ MỚI

Manual review không thể scale

GIAI ĐOẠN ĐẦU

~100 traces / tuần

- ✓ Hiểu được patterns
- ✓ Manual review khả thi

SAU KHI SCALE

100k production traces

- ✕ Không thể review hết
- ✕ Không thể chậm mỗi release

AUTOMATION MUA LẠI

Ship nhanh hơn — chạy eval thay vì tranh luận

Bắt drift và regression trước khi user thấy

Đo được → cải thiện được

⚠ **Build quá sớm** → mất thời gian đo sai thứ

⚠ **Build sai** → false confidence, system fail âm thầm

Không phải vấn đề nào cũng cần eval

Ba câu hỏi trước khi viết eval cho một trace category — chỉ **generalization gap** mới đáng tự động hoá.

CÂU HỎI 1 · SPECIFICATION CHECK

Prompt có được giao xử lý tình huống này không?

KHÔNG →

Specification gap — thiếu đặc tả

Sửa system prompt — chưa cần viết eval.

Email drafter không tạo subject line vì prompt chưa hề yêu cầu.

↓ Có, prompt đã rõ

CÂU HỎI 2 · SYSTEM CHECK

Agent có bao giờ làm được việc này không?

KHÔNG BAO GIỜ →

Architectural issue — thiếu năng lực hệ thống

Engineering fix — thiếu tool, integration, hoặc model capability.

Calendar API chưa connect — prompt rõ đến đâu cũng không làm được.

↓ Có lúc làm được

CÂU HỎI 3 · GENERALIZATION CHECK

Lúc được lúc không — dù prompt đã rõ?

CÓ →

Generalization gap — thiếu tính nhất quán

Viết automated eval ✓ — đây là ứng viên phù hợp.

Email drafter đôi khi cá nhân hoá đúng, nhưng chỉ ở case rất rõ ràng.

Chọn cách đánh giá: Code-based hay LLM-as-Judge?

Nếu có thể kiểm tra bằng code — hãy dùng code trước.

CODE-BASED — MẶC ĐỊNH ƯU TIÊN

Nhanh hơn | Rẻ hơn | Ổn định | Dễ gắn critical path | Dễ debug

CODE LÀM TỐT

- Có đúng format không
- Có thiếu trường bắt buộc không
- Có gọi đúng tool không
- Có lộ thông tin cấm không
- Có trả lời sai schema không

LLM-AS-JUDGE — KHI CODE KHÔNG ĐỦ

Dùng khi tiêu chí phụ thuộc vào sắc thái ngôn ngữ và ngữ cảnh — không thể viết thành rule rõ ràng.

CODE KHÔNG LÀM TỐT

- Câu trả lời có hữu ích với đúng người dùng không?
- Explanation có hợp lý và nhất quán không?
- Cách diễn đạt có đủ thuyết phục không?
- Output có đúng brand voice không?

Rule rõ, kiểm được bằng code → dùng code Cần sắc thái, ngữ cảnh → LLM Judge

Tiêu chí nào giao được cho máy? Hỏi một câu duy nhất

> "Tiêu chí giao được cho máy khi và chỉ khi nó có một **referent kiểm chứng được** — một thứ bên ngoài phán đoán của model mà kết luận có thể quy về."

Tài liệu nguồn → chấm được tính đúng

Bộ quy tắc rõ ràng → chấm được hình thức

Referent = "phán đoán của người" → không bao giờ giao máy chấm

REFERENT RÕ, TÁCH NHỎ ĐƯỢC

Máy chấm — người kiểm mẫu

Máy ra quyết định; người kiểm ~10% ngẫu nhiên để canh drift.

REFERENT MỘT PHẦN

Máy sàng lọc — người thẩm định

Máy xếp hạng nghi ngờ và nêu bằng chứng; người quyết mọi case bị cờ.

KHÔNG CÓ REFERENT

Người quyết — máy nêu bằng chứng

Máy không được xuất điểm số — chỉ xuất dữ kiện trung gian cho người xem.

Tổng hợp từ khảo sát 239 tài liệu về LLM-as-a-judge (2024–2026)

PART 05

Code-based Evaluation

Deterministic checks chạy tự động trên mọi prompt change — nền tảng không thể thương lượng của eval suite.

PYTHON · DETERMINISTIC · KHÔNG GỌI LLM

Ba phần của một code-based eval

```
① INPUT
def check(output_str, user_input):
② LOGIC
  if "Subject:" not in output_str:
③ RESULT + REASON
    return False, "Missing subject line"
  return True, None
```

① Input — lấy đúng phần cần kiểm tra

Output string, JSON, tool calls — hoặc kết hợp cả user input khi cần kiểm tra theo ngữ cảnh.

② Logic — rule đủ rõ để máy quyết định

Condition, pattern match, schema validation — nếu viết được "phải có X" hay "đúng format Z" thì dùng code.

③ Result — pass/fail kèm reason string

Reason string không phải "có thì tốt" — khi fail ở trace 38/50, team phải biết ngay tại sao, không cần mở lại trace để đoán.

Category checking — Support Triage Agent

Agent phải chọn đúng 1 trong 3 labels: Technical · Billing · Feature Request

```
VALID = {"Technical", "Billing", "Feature Request"}

def check_category_label(output: str) → dict:
  found = [c for c in VALID if c in output]

  if len(found) == 1:
    return {"pass": True,
      "reason": f"Single valid: {found[0]}"}
  elif len(found) == 0:
    return {"pass": False,
      "reason": "No valid category found"}
  else:
    return {"pass": False,
      "reason": f"Multiple labels: {found}"}
```

Danh sách labels được phép

Chỉ 3 giá trị exact string. “billing issue” hay “tech problem” không được tính.

Đếm bao nhiêu label xuất hiện

Quét output, gom tất cả label tìm thấy — rỗng, 1 phần tử, hoặc nhiều hơn.

Ba trường hợp, một kết quả rõ ràng

pass	Đúng 1 → ghi rõ label nào
fail	Không có → “No valid category found”
fail	Nhiều hơn 1 → liệt kê cụ thể để debug

Bốn nơi nên bắt đầu với code-based evals

Output có đúng cấu trúc không?	STRUCTURE / FORMAT	Đủ field, đúng format, đúng schema, trong giới hạn cho phép. Quan trọng khi output đi tiếp vào UI, database hoặc downstream system.
Output có đủ phần bắt buộc không?	PRESENCE / COVERAGE	Có nhắc đúng product name, ticket ID, required policy, source, next step? Bắt những output nghe ổn nhưng chưa đủ để user hành động.
Agent có đi đúng quy trình không?	TOOL-CALL / SEQUENCING	Gọi đúng tool, đúng thứ tự, đúng parameter. Nhiều lỗi không nằm ở câu trả lời cuối mà nằm ở cách agent thực thi workflow.
Chỉ số vận hành có trong ngưỡng không?	THRESHOLD CHECKS	Latency, cost, token count, confidence score có vượt ngưỡng không? Trả lời đúng nhưng chậm hơn hoặc đắt hơn nhiều vẫn là regression.

Output phải dùng được · nội dung phải đủ ý · quy trình phải đúng · chi phí phải trong ngưỡng.

Case study — ba kiểm tra tự động cho Support Triage Agent

KIỂM TRA 1

Câu trả lời có dùng đúng nhãn chuẩn không?

Phải chọn đúng một trong ba nhãn. Diễn đạt lại sẽ làm hỏng hệ thống phân loại phía sau.

✓ Đúng
Danh mục: Billing — khách khiếu nại hoá đơn tháng 3.

× Không tìm thấy nhãn hợp lệ
“Đây có vẻ là vấn đề liên quan đến thanh toán.”

KIỂM TRA 2

Agent có bịa thông tin không có trong yêu cầu không?

So sánh mã ticket trong câu trả lời với mã trong yêu cầu ban đầu.

✓ Đúng
Yêu cầu có TKT-00421 · câu trả lời chỉ nhắc TKT-00421.

× Bịa ra mã TKT-00422
Yêu cầu có TKT-00421 · câu trả lời nhắc TKT-00422.

KIỂM TRA 3

Thời gian phản hồi có còn trong ngưỡng không?

Câu trả lời đúng nhưng chậm hơn nhiều vẫn là lỗi cần bắt.

✓ Đúng
Thời gian 1.1 giây · ngưỡng tối đa 2.0 giây.

× Vượt ngưỡng 2.0 giây
Thời gian 2.4 giây.

Sau khi cập nhật prompt: cả ba pass → mới xem xét release.

Đây là mức sàn tối thiểu — câu hỏi tinh tế hơn vẫn cần người đọc và đánh giá.

Đọc kết quả eval trên dataset

Pass rate tổng là bao nhiêu?
So với phiên bản production

Failures tập trung ở đâu?
Nhóm input nào đang fail nhiều?

Lỗi có đi cùng nhau không?
Row nào fail nhiều evals cùng lúc?

Kết quả có bất thường không?
Pass rate = 0% hoặc quá thấp

TÍN HIỆU TỐT	CẦN XEM LẠI
✓ Pass rate bằng hoặc cao hơn baseline	× Pass rate tụt đáng kể dù spot check trông ổn
✓ Failures rải đều — không nhóm nào nổi bật	× 80%+ failures từ cùng một loại input → failure mode cụ thể
✓ Failures phân tán, không row nào fail nhiều evals	× Cùng row fail 2+ evals → cụm input khó, không phải lỗi rời rạc
✓ Kết quả nằm trong range hợp lý, khớp expectation	**Pass rate = 0% → kiểm tra eval trước!** Regex sai, field sai — đừng vội điều tra agent.

Không cần 100% — pass rate là một quyết định sản phẩm

Cùng một con số 90% — là thành công với sản phẩm này, là thảm hoạ với sản phẩm khác.



NGƯỠNG PHỤ THUỘC | Chi phí của một lỗi | Có human review phía sau không | Ai là người chịu rủi ro

NỐI VÀO BUSINESS METRIC

Factual consistency **80%** → tự động hoá **30%** request · **90%** → **50%** · **98%** → **90%** — Chip Huyen, AI Engineering ch.4

Kèm **usefulness threshold**: dưới ngưỡng nào đó (vd 50%) thì vô dụng với mọi loại request — đó là sàn, không phải mục tiêu.

Đưa code evals vào quy trình release

NGUYÊN TẮC 1

Eval là release gate, không phải kiểm tra tuỳ hứng

Mỗi prompt change hoặc model upgrade chạy lại eval tự động. Mọi thay đổi đi qua cùng một tiêu chuẩn — không chờ đến sát release.

Pass → Ship

Fail → Dừng

NGUYÊN TẮC 2

Chốt ngưỡng trước khi bắt đầu iterate

Quyết định sau khi thấy số thì không còn là tiêu chuẩn. Viết ngưỡng ra trước: nhãn >90%, 0 lỗi bịa nghiêm trọng, latency <2s.

"78% cũng ổn mà" — không phải threshold, là thương lượng.

Verdict phải "có răng" — mỗi kết luận kèm một kế hoạch

"Ship" hay "chưa ship" chỉ là một chữ — giá trị nằm ở việc nó buộc bạn làm gì tiếp theo.

✓ Ship

...kèm kế hoạch monitoring tuần đầu

Sample **1–10%** session production để chấm tiếp.

Nhìn 3 tín hiệu drift: offline–online lệch dần · failure mode mới · user feedback mâu thuẫn eval score.

Ngưỡng alert cụ thể — không phải "để ý xem sao".

⟳ Iterate

...kèm nhánh hành động — đòn rẻ nhất trước

1 · Prompt — siết instruction, thêm near-miss. Rẻ nhất.

2 · Đổi model — chạy lại cả suite, kèm latency SLA (ngưỡng thời gian phản hồi cam kết).

3 · Architecture — thêm tool, tách bước. Đắt nhất, làm cuối.

✕ Revert

...khi không có cải thiện đo được

Thay đổi "cảm giác đáng lẽ giúp" mà số không nhích — không phải cải thiện.

Quay về baseline, ghi vào experiment log.

Đừng giữ thay đổi vì đã lỡ tốn công làm nó.

Quy tắc ship: mọi eval đạt ngưỡng và **không eval nào tụt quá 2%** so với baseline — nhớ: trên dataset nhỏ, chênh ≤2% là nhiễu.

PART 06

LLM Judge Calibration

Hiệu chỉnh bộ chấm để biết nó có đúng, không chỉ biết nó nghĩ gì.

LLM Judge — model thứ hai chấm output của model thứ nhất

Cho những tiêu chí code không kiểm được: tone, grounding, tính nhất quán của lập luận.

INPUT · THỨ CẦN CHẤM

Trace: khách bực vì giao trễ → agent trả lời "Tôi thấy đơn của bạn bị trễ. Quy trình hoàn tiền như sau…"

↓

+ TIÊU CHÍ & VÍ DỤ BOUNDARY

"Pass = có ghi nhận cảm xúc TRƯỚC khi xử lý. Nhắc đến sự việc ≠ ghi nhận cảm xúc." + 1 ví dụ pass sát ranh, 1 ví dụ fail sát ranh

↓

JUDGE — MODEL KHÁC HỌ

Fail

Reasoning: "Agent nhắc sự việc nhưng đi thẳng vào quy trình — không có biểu hiện quan tâm."

1 Mỗi judge — một tiêu chí

Judge "chấm chất lượng tổng thể" không align được với ai.

2 Verdict nhị phân, không thang điểm

Thang 1–10 đắt để align — LLM dồn về giữa thang để né quyết định khó.

3 Kèm ví dụ sát ranh giới

Một pass sát ranh + một fail sát ranh dạy judge ranh giới nằm đâu.

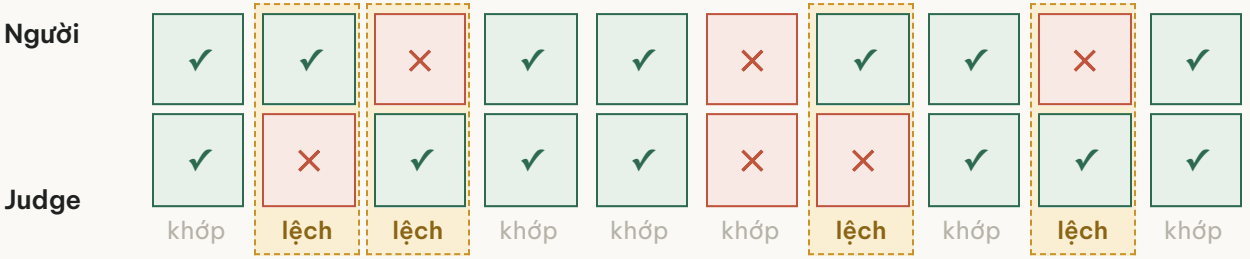
4 Judge khác họ model với generator

Chấm bài "người nhà" → xác suất cho qua lỗi cao hơn 50% (Pombal 2026).

Verdict của judge là một ý kiến — không phải sự thật. Đo độ tin của ý kiến đó chính là calibration.

Pass rate giống nhau — không có nghĩa judge nghĩ giống bạn

Calibration = so phán quyết của judge với nhãn của expert trên cùng một bộ trace, từng dòng một. Ví dụ: 10 trace, hai bên cùng chấm 7 Pass — pass rate y hệt 70%.



CÙNG 10 TRACE ĐÓ ĐỔ XUỐNG MA TRẬN ▼

	THẬT SỰ TỐT (7)	THẬT SỰ XẤU (3)
Judge nói TỐT	5 ✓ **Đúng** — Nhận ra đúng output tốt	2 ⚠ **Cho qua lỗi — nguy hiểm nhất** — Team tưởng ổn, lỗi lọt production
Judge nói XẤU	2 ⚠ **Chặn nhầm — tốn công** — Đi sửa thứ không phải vấn đề thật	1 ✓ **Đúng** — Bắt đúng output xấu

ĐỒNG THUẬN THẬT TỪNG DÒNG

6/10

TPR = 5/7 = 71% — gặp output tốt, nhận đúng mấy lần. Thấp → chặn nhầm, điều tra oan.

TNR = 1/3 = 33% — gặp output lỗi, bắt được mấy lần. Thấp → cho qua lỗi, nguy hiểm nhất.

Judge chưa calibrate thường **quá dễ dãi** — pass rate nói judge **nghĩ gì**, calibration mới nói judge có nghĩ **giống bạn** không.

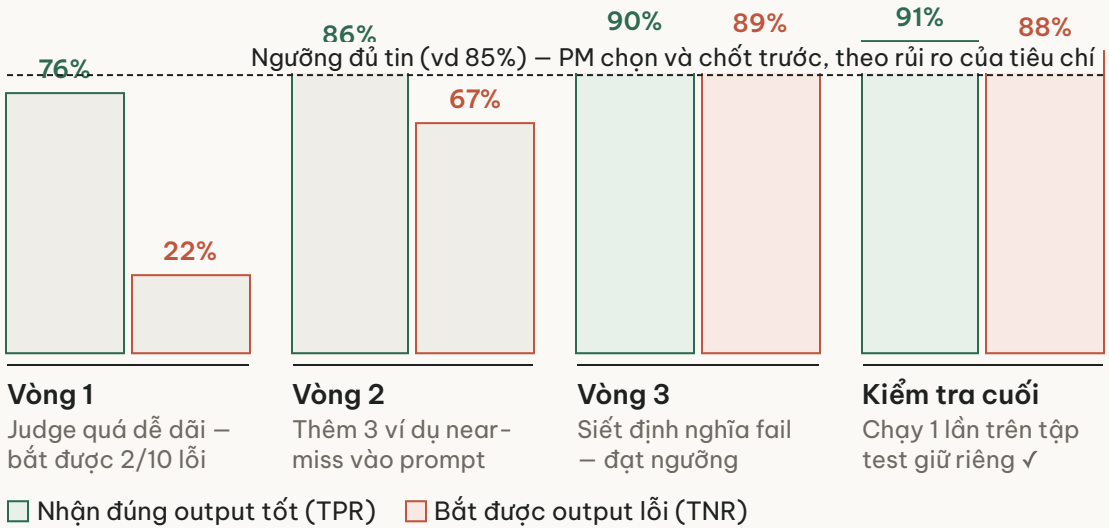
Calibration là một vòng hiệu chỉnh — bắt đầu chưa hoàn hảo là bình thường

Không ai viết được judge đúng ngay từ đầu. Việc của bạn là lặp vòng cho đến khi đạt ngưỡng đủ tin.

MỘT VÒNG HIỆU CHỈNH

1. Judge chấm 50 case đã có nhãn expert
2. So từng dòng — đo TPR / TNR
3. Đọc các case hai bên lệch nhau
4. Sửa prompt judge — thêm ví dụ near-miss

↻ Lặp — mỗi vòng ~nửa ngày, thường 2–3 vòng là đạt



Cái được phép chưa hoàn hảo là **judge** — cái phải có ngay từ đầu là **nhãn expert** để đo nó.

Ba sự thật khó chịu về LLM judge

Judge dễ dãi, không phải nhiễu

96% nhận đúng output tốt

<25% bắt được output lỗi

Rủi ro là **cho qua lỗi** — đúng ô nguy hiểm nhất của confusion matrix.

Jain 2025

"Khớp 85%" nghe cao — phần lớn là khớp nhờ may rủi

Dataset nhiều output tốt → hai bên cùng Pass là chuyện dễ trùng nhau. Trừ phần trùng-do-may-rủi đi, "khớp 85%" thường chỉ còn **κ ≈ 0.45–0.52** — mức hiểu nhau trung bình yếu.

Judge lặp lại rất ổn định — **cùng một sai lầm**. Ổn định không phải là đúng.

κ (kappa): 0 = ngang đoán bừa · 1 = khớp tuyệt đối. Khoảng thổi phồng đo được: 33–41 điểm (Norman 2026).

Norman 2026 · 541.000 phán đoán, 21 judge

Thiên vị "người nhà"

Judge có xác suất cho qua lỗi **của chính họ model mình** cao hơn trên 50%.

Quy tắc: **generator và judge phải khác họ model**.

Pombal 2026

Hệ quả cho hội đồng nhiều judge: **đừng lấy đa số phiếu** — đa số thừa hưởng sự dễ dãi. Một model gắn cờ là đủ để đưa case cho người xem.

Sáu bước calibration

Bước 4–5 là một vòng lặp — chỉ thoát khi TPR/TNR đạt ngưỡng; bước 6 chạy đúng một lần.

1
Label 50–100 trace
Expert chấm Pass/Fail đúng tiêu chí judge sẽ chấm — chia dev / test.

→

2
Chạy judge trên dev set
Ghi verdict + reasoning cạnh nhãn người.

→

3
Dựng confusion matrix
TPR: gặp output tốt, judge nhận đúng mấy lần · TNR: gặp output lỗi, judge bắt được mấy lần.

bước bị làm ẩu nhất — đọc TỪNG bất đồng

VÒNG LẶP — ĐẾN KHI ĐẠT NGƯỠNG

4
Đọc TỪNG bất đồng
Phân loại: hiểu sai tiêu chí · thiếu ví dụ boundary · leniency bias.

⟳ lặp

5
Sửa prompt — một thứ mỗi lần
Đòn bẩy nhất: thêm 3–4 near-miss example (TNR +15–25 điểm).

→

6
Validate test set — MỘT lần
Chênh dev ≤ 5 điểm → calibrated. Ghi TPR, TNR, prompt, ngày.

CASE EMPATHY JUDGE — R1 · 76 / 22 — vòng đầu thế là bình thường → R2 · 86 / 67 → R3 · 90 / 89 → **Test · 88 / 85 — deploy** (TPR / TNR, %)

NEXT ACTION

- **TNR thấp** → siết fail criteria + thêm 3–4 near-miss
- **TPR thấp** → thêm ví dụ Pass sát ranh + mục "không bắt buộc"
- **Cả hai ì** → tách judge hẹp hơn · đổi model · đo trần người
- **<2–3 điểm/vòng** → chạm trần: đổi vai judge, người quyết

Calibrate judge không cần hạ tầng — một spreadsheet là đủ

calibration_round_3.xlsx

OUTPUT CỦA AGENT	JUDGE	EXPERT
Hoàn tiền đúng chính sách, giải thích rõ	✓	✓
Đúng chính sách nhưng bỏ sót câu hỏi thứ hai	✓	✗
Từ chối một yêu cầu hoàn toàn hợp lệ	✗	✗

▪ Dòng bất đồng — nơi duy nhất đáng đào sâu

1 Gửi expert 25–50 case, vài ngày một lần

Kèm cả verdict và lý do của judge để expert phản biện.

2 Chỉ đào sâu chỗ hai bên bất đồng

Bất đồng cho biết judge hiểu sai tiêu chí ở đâu — hoặc tiêu chí đang mơ hồ.

3 Sửa prompt của judge, đo lại vòng sau

Theo dõi tỉ lệ đồng thuận tăng dần qua từng vòng — ngay trong spreadsheet.

Tham chiếu: Hamel Husain — Your AI Product Needs Evals

Khi nào LLM Judge đã chạm trần?

Nhận ra thời điểm này quan trọng — tránh tốn thời gian tối ưu thứ không còn cải thiện được.

DẤU HIỆU 1

Model không thật sự hiểu domain

Reasoning strings cho thấy model thiếu nền tảng. Ví dụ: đánh giá legal citation hay medical recommendation — model không có năng lực ở domain đó.

Vấn đề là model capacity — không phải prompt.

DẤU HIỆU 2

Thêm prompt mà chất lượng gần như không nhích

Effort tăng nhưng improvement hầu như bằng không qua nhiều vòng. Đã rút gần hết giá trị có thể lấy từ model này.

Đã chạm điểm trần với model này.

DẤU HIỆU 3

Ngay cả humans cũng chưa thống nhất

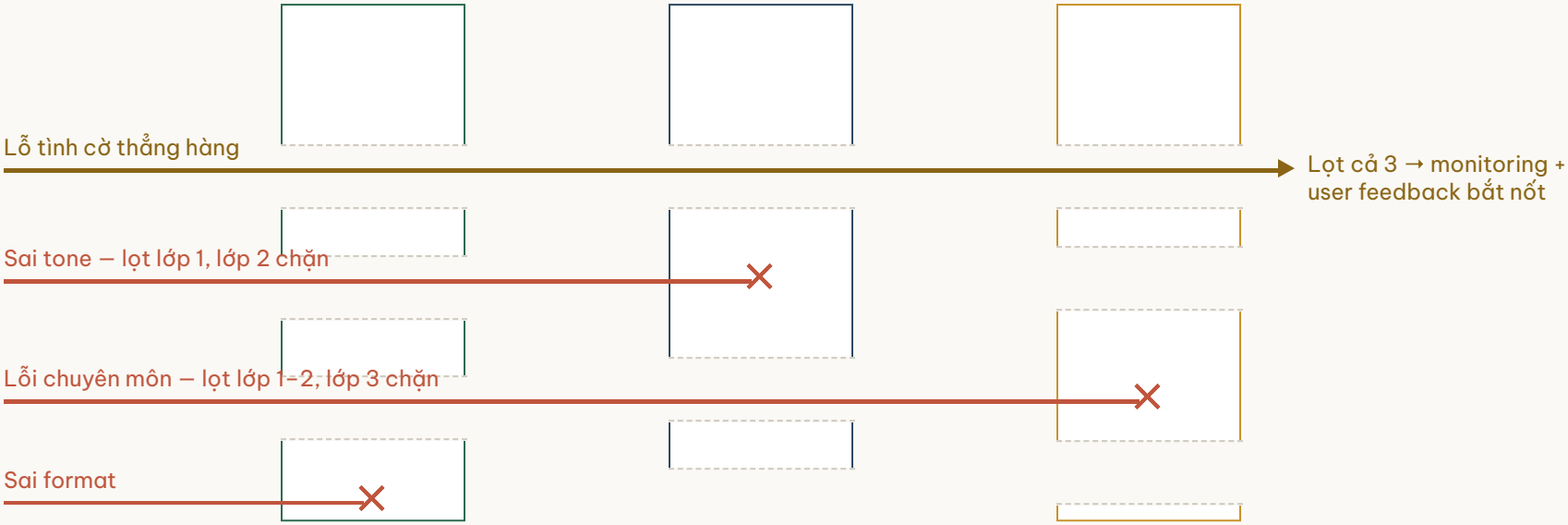
Trên tiêu chí cảm tính, chính các expert chỉ đồng thuận với nhau ở mức ICC 0.07–0.18 — hệ số đồng thuận giữa người chấm, 0 = không khớp, 1 = tuyệt đối (Linde 2026). Judge đạt 0.30 ở đó không hề kém người — chính tiêu chí đang hỏng.

Đo trần người-người trước — bất đồng cao giữa người là lỗi đặc tả rubric, không phải cơ hội tự động hoá.

Vấn đề không còn nằm ở cách viết prompt — mà ở giới hạn của chính model hoặc tiêu chí đang được đánh giá.

Không lớp nào bắt được mọi lỗi — nên xếp lớp

Mỗi lớp đều có lỗ, nhưng lỗ không trùng nhau — lỗi lọt lớp này bị lớp sau chặn.



CHI PHÍ QUYẾT ĐỊNH TẦN SUẤT

L1 Code evals — chạy liên tục

L2 Human & judge — định kỳ

L3 A/B testing — sau thay đổi lớn

Swiss cheese model — Anthropic · Hamel

PART 07

Đưa expert vào loop

Bằng chứng thay điểm số · Remove friction · Label là tài sản

Cho expert xem bằng chứng, đừng đưa sẵn điểm số

KHI MÁY GỢI Ý SẴN KẾT LUẬN

+2.9 điểm chính xác khi AI gợi ý đúng

−11.3 điểm khi AI gợi ý lệch — người bị kéo sai theo

Downside gấp ~**4 lần** upside — điểm số gợi ý sẵn là can thiệp rủi ro cao.

Jabbour 2023 · RCT — thử nghiệm đối chứng ngẫu nhiên trên 457 bác sĩ

MÁY XUẤT ĐIỂM SỐ

Tăng chi phí **bất đồng** — muốn phản bác phải tự dựng lập luận.

"Độ tin cậy: 0.86" → trượt

MÁY XUẤT BẰNG CHỨNG

Giảm chi phí **kiểm tra** — chỉ cần liếc đoạn được trích.

"Trích chính sách, mục 4.2: hoàn nếu trễ > 3 ngày" → đạt

Phép thử thiết kế: **"Output này có giúp expert xác nhận hay bác bỏ nhanh hơn so với không có nó không?"**

THỨ TỰ MÀN HÌNH CHỐNG LỆCH Bằng chứng → Người phán đoán → Mới hiện điểm máy Fogliato 2022

“Remove ALL friction” — để expert nhìn data không tốn sức

Review một trace phải mở 4 tab → không ai review đủ nhiều → không ai biết hệ thống fail kiểu gì.

1 Một màn hình đủ ngữ cảnh

Trace, dữ liệu liên quan, chính sách — gom về một chỗ, không bắt mở tab.

2 Ngôn ngữ domain, giấu jargon AI

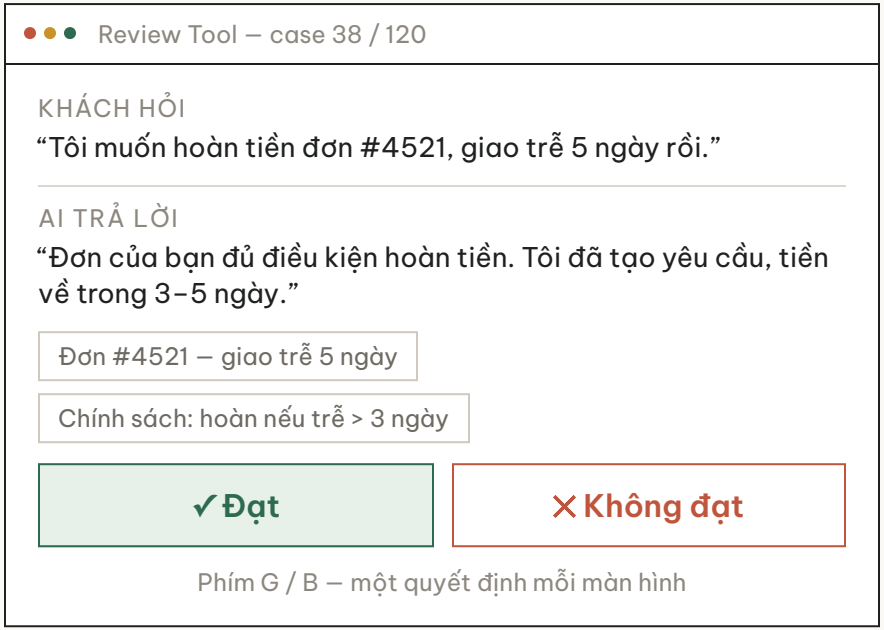
Expert không cần biết trace, span, tool call là gì để chấm được.

3 Một quyết định nhị phân

Đạt / Không đạt — thang điểm chi tiết chậm hơn và khó quản lý hơn nhiều.

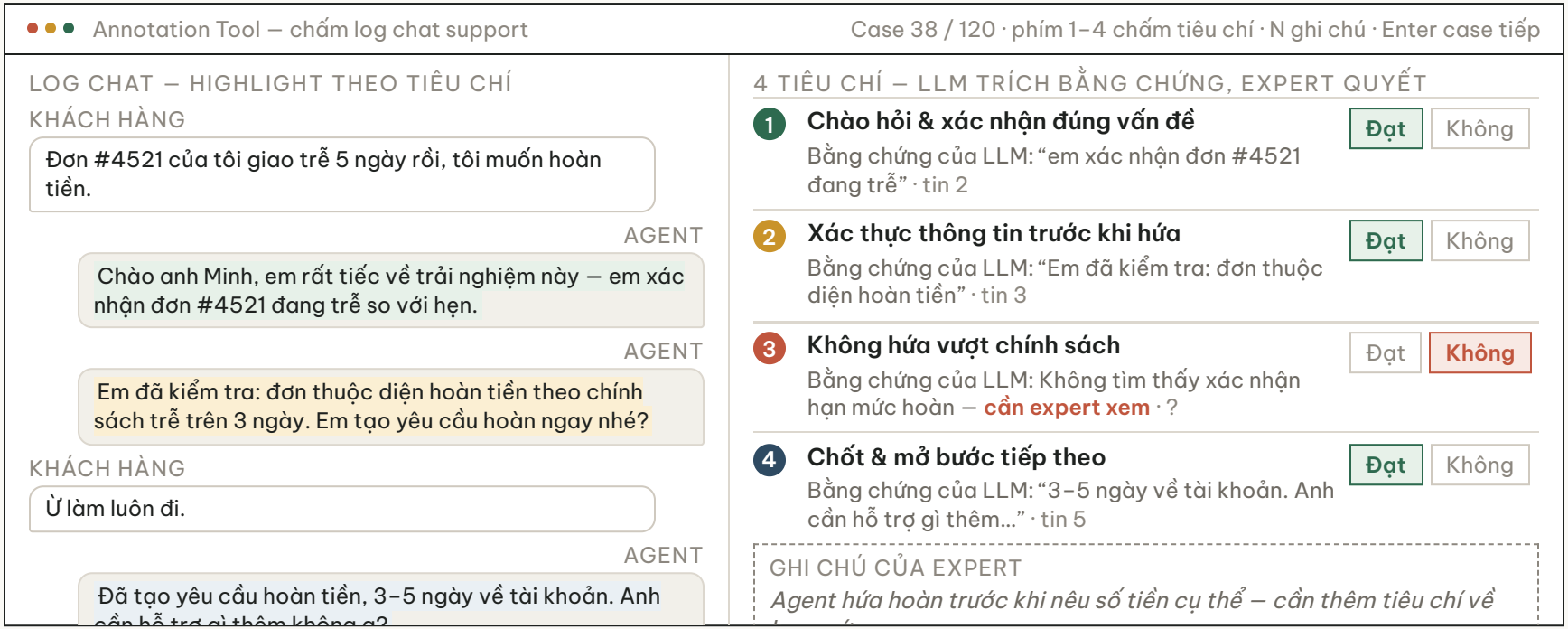
4 Build trong ~1 ngày

Đây không phải dự án hạ tầng — một tool nội bộ nhỏ, PM yêu cầu được ngay.



Khi máy không chấm được — build luồng LLM hỗ trợ expert chấm

LLM đọc trước, trích bằng chứng cho từng tiêu chí — expert chỉ validate. Ví dụ: chấm log chat CSKH theo 4 tiêu chí.



VAI CỦA LLM Ở ĐÂY | Trích bằng chứng + trỏ vị trí | Không xuất điểm số | Verdict thuộc về expert

Dùng được cho mọi artifact: log chat, video theo chuẩn, tài liệu...

Cùng một buổi chấm của expert — dùng lại được ba lần

Ví dụ: expert bỏ 2 giờ, chấm 300 case Đạt / Không đạt trên tool review.

ĐẦU VÀO — 2 GIỜ CỦA EXPERT

300 case

đã chấm Đạt / Không đạt
234 đạt · 66 không đạt

→

1 Đo chất lượng thật của hệ thống

Đếm nhãn: 234/300 đạt — con số thay cho cảm giác "có vẻ ổn".

→ Baseline: hệ thống đang ở 78%

→

2 Calibrate LLM judge

Chạy judge trên cùng 300 case, so từng dòng với nhãn expert.

→ Judge chỉ khớp 61% — biết chính xác chỗ sửa prompt

→

3 Curate dataset

66 case Không đạt → test chống tái phát; 12 case xuất sắc → chuẩn mới.

→ Regression suite + golden outputs mới

"Một giờ review của expert — nếu được thiết kế đúng — là tài sản dùng lại ba lần."

Hệ thống evals hoàn chỉnh trông như thế này

Bắt đầu từ use case, kết thúc ở quyết định ship — mỗi tiêu chí có đúng bộ chấm của nó.

#	BƯỚC	INPUT	OUTPUT
1	**Dataset có chủ đích**	Use case + 3–5 dimension (grid)	40–50 test input phủ đủ ô grid
2	**Chạy agent, đọc trace**	40–50 test input	Trace codes — taxonomy lỗi của chính sản phẩm
3	**Rubric nhị phân**	Trace codes	Tiêu chí Pass/Fail — người chấm đồng thuận >90%
4	**Route bộ chấm theo referent**	Từng tiêu chí trong rubric	Tiêu chí nào → Code · LLM Judge · Human
5	**Calibrate judge**	50–100 nhãn expert (golden labels)	TPR/TNR đạt ngưỡng — "giấy phép" của judge
6	**Chạy full → đọc report**	Mọi thay đổi prompt / model	Bảng eval report — cầm đi ra quyết định →

Eval Report — candidate v1.4 vs baseline v1.3

TIÊU CHÍ	BỘ CHẤM	KẾT QUẢ
Đúng format & nhãn chuẩn	Code	98% (+2) ✓
Không bịa ticket ID	Code	100% (=) ✓
Grounding — bám nguồn	Judge · TPR 88/TNR 85	86% (+2) ✓
Empathy	Judge · đã calibrate	79% (−1) → đọc trace
Chuẩn chuyên môn	Human · sample 10%	92% ✓

Coverage: **18/20 ô grid** có mặt trong dataset

Ship + monitoring tuần đầu

Tổng kết

Đo công việc AI làm ra — PM là người định nghĩa "tốt"

- Đọc trace là user research mới — đừng tin lời agent nói, kiểm trạng thái cuối
- Thu hẹp scope đến khi viết được rubric nhị phân cả team chấm giống nhau
- Dataset: người cầm coverage, LLM chỉ diễn đạt — phủ độ khó, không phủ tần suất

Tự động hoá đúng thứ đáng tự động hoá

- Code là mặc định — judge cho sắc thái, và chỉ tin judge sau khi calibrate
- Judge dễ dãi hơn bạn tưởng — so với trần người-người, không so với 1.0
- Expert vào loop: bằng chứng thay điểm số, một nhãn dùng lại ba lần

AI Evals

Model mới ra sáng mai — team bạn có dám ship trong 24 giờ không?